

Analysis of Machine Learning Models for Loan Prediction

This report presents the development and evaluation of six machine learning algorithms for predicting loan approval status, categorized as approved or not approved. The dataset consisted of an equal distribution of approved and not approved samples (50% each). Each model was evaluated using accuracy, precision, recall, F1-score, and AUC metrics to ensure comprehensive performance assessment. The best-performing algorithm was selected based on balanced performance, with special emphasis on F1-score and AUC, to provide reliable predictions for both classes.

Algorithms Used:

- Logistic Regression
- Support Vector Machine (SVM)
- Decision Tree
- K-Nearest Neighbors (KNN)
- Random Forest
- Multi-Layer Perception (MLP) Neural Network

1. Logistic Regression

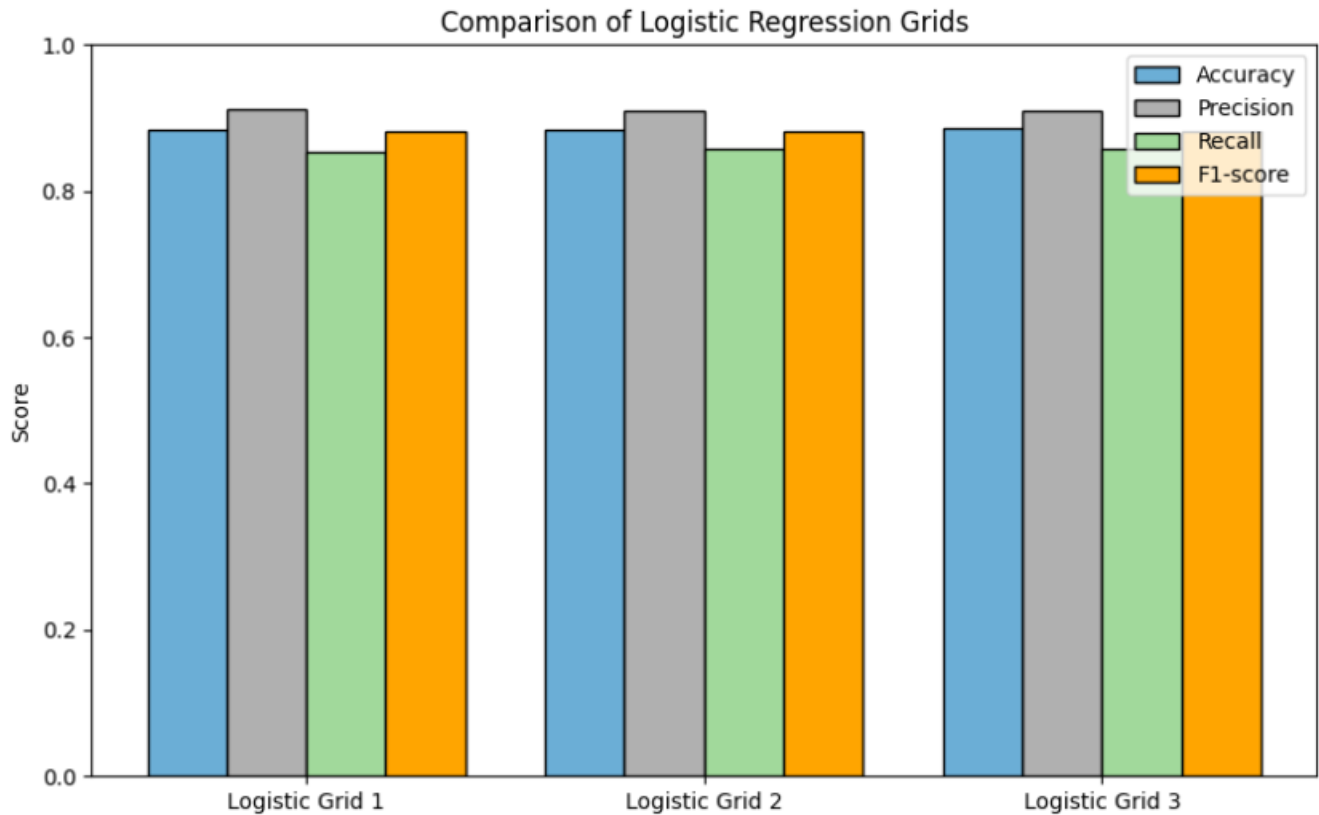
Used Hyperparameters:

```
# Grid 1
param_grid_log_1 = {
    'C': [0.01, 0.1],
    'max_iter': [100, 200],
    'penalty': ['l2'],
    'solver': ['liblinear']
}

# Grid 2
param_grid_log_2 = {
    'C': [1, 10],
    'max_iter': [500, 1000],
    'penalty': ['l2'],
    'solver': ['liblinear']
}

# Grid 3
param_grid_log_3 = {
    'C': [50, 100],
    'max_iter': [1500, 2000],
    'penalty': ['l1'],
    'solver': ['saga']
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Grid 1	0.884429	0.911394	0.853146	0.881309
1	Logistic Grid 2	0.884714	0.908721	0.856839	0.882018
2	Logistic Grid 3	0.885071	0.908790	0.857549	0.882426



Step 1: Look at F1-score (most balanced metric for class imbalance)

- Logistic Grid 1 → 0.881309
- Logistic Grid 2 → 0.882018
- Logistic Grid 3 → 0.882426 (**highest F1-score**)

Step 2: Check Accuracy

- Logistic Grid 1 → 0.884429
- Logistic Grid 2 → 0.884714
- Logistic Grid 3 → 0.885071 (**slightly higher than the others**)

➤ **Logistic Grid 3** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

2. Support Vector Machine (SVM)

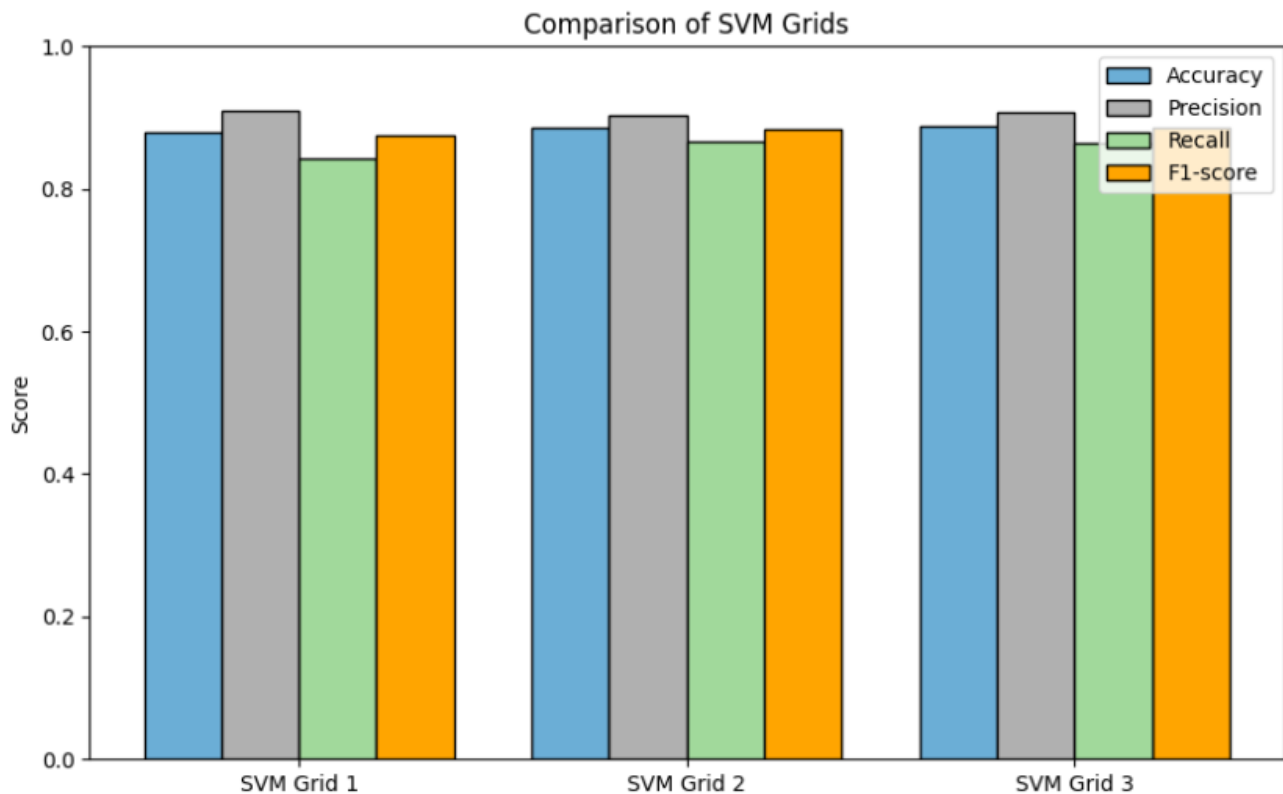
Used Hyperparameters:

```
# Grid 1
param_grid_svm_1 = {
    'C': [0.01, 0.1],
    'kernel': ['linear'],
    'gamma': ['scale']
}

# Grid 2
param_grid_svm_2 = {
    'C': [1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

# Grid 3
param_grid_svm_3 = {
    'C': [50, 100],
    'kernel': ['rbf'],
    'gamma': ['scale', 'auto']
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	SVM Grid 1	0.879143	0.910310	0.841813	0.874722
1	SVM Grid 2	0.886857	0.903461	0.866895	0.884800
2	SVM Grid 3	0.887929	0.908396	0.863474	0.885366



Step 1: Look at F1-score (most balanced metric for class imbalance)

- SVM Grid 1 → 0.874722
- SVM Grid 2 → 0.884800
- SVM Grid 3 → 0.885366 (**highest F1-score**)

Step 2: Check Accuracy

- SVM Grid 1 → 0.879143
- SVM Grid 2 → 0.886857
- SVM Grid 3 → 0.887929 (**slightly higher than the others**)

➤ **SVM Grid 3** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

3. Decision Tree

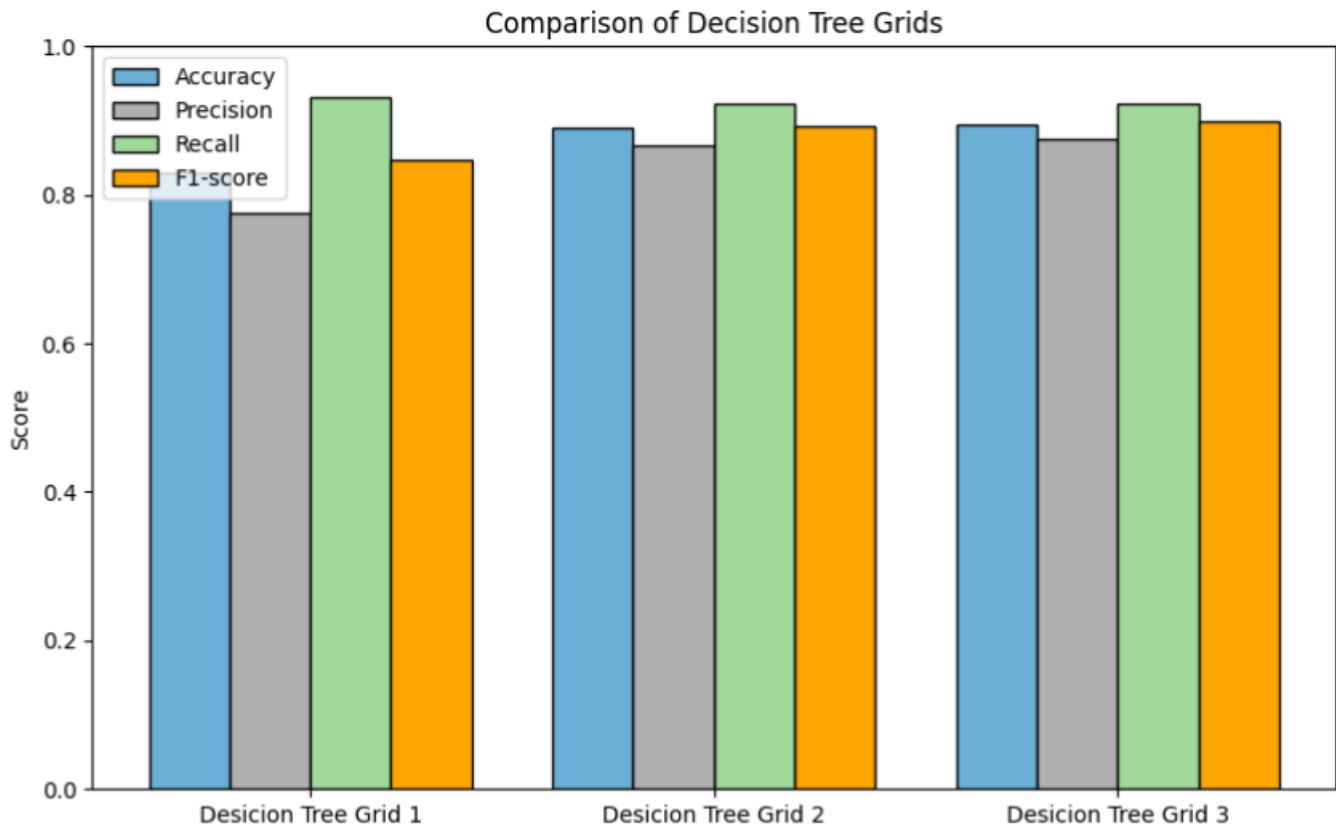
Used Hyperparameters:

```
# Grid 1
param_grid_dt_1 = {
    'max_depth': [3],
    'min_samples_split': [10],
    'criterion': ['gini']
}

# Grid 2
param_grid_dt_2 = {
    'max_depth': [5, 10],
    'min_samples_split': [2, 5],
    'criterion': ['gini', 'entropy']
}

# Grid 3
param_grid_dt_3 = {
    'max_depth': [None, 15, 20],
    'min_samples_split': [2, 3],
    'criterion': ['entropy', 'gini']
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	Desicion Tree Grid 1	0.830214	0.775012	0.931737	0.846179
1	Desicion Tree Grid 2	0.889643	0.865775	0.922902	0.893426
2	Desicion Tree Grid 3	0.894857	0.874612	0.922474	0.897905



Step 1: Look at F1-score (most balanced metric for class imbalance)

- Decision Tree Grid 1 → 0.846179
- Decision Tree Grid 2 → 0.893426
- Decision Tree Grid 3 → 0.897905 (**highest F1-score**)

Step 2: Check Accuracy

- Decision Tree Grid 1 → 0.830214
- Decision Tree Grid 2 → 0.889643
- Decision Tree Grid 3 → 0.894857 (**slightly higher than the others**)

➤ **Decision Tree Grid 3** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

4. K-Nearest Neighbors (KNN)

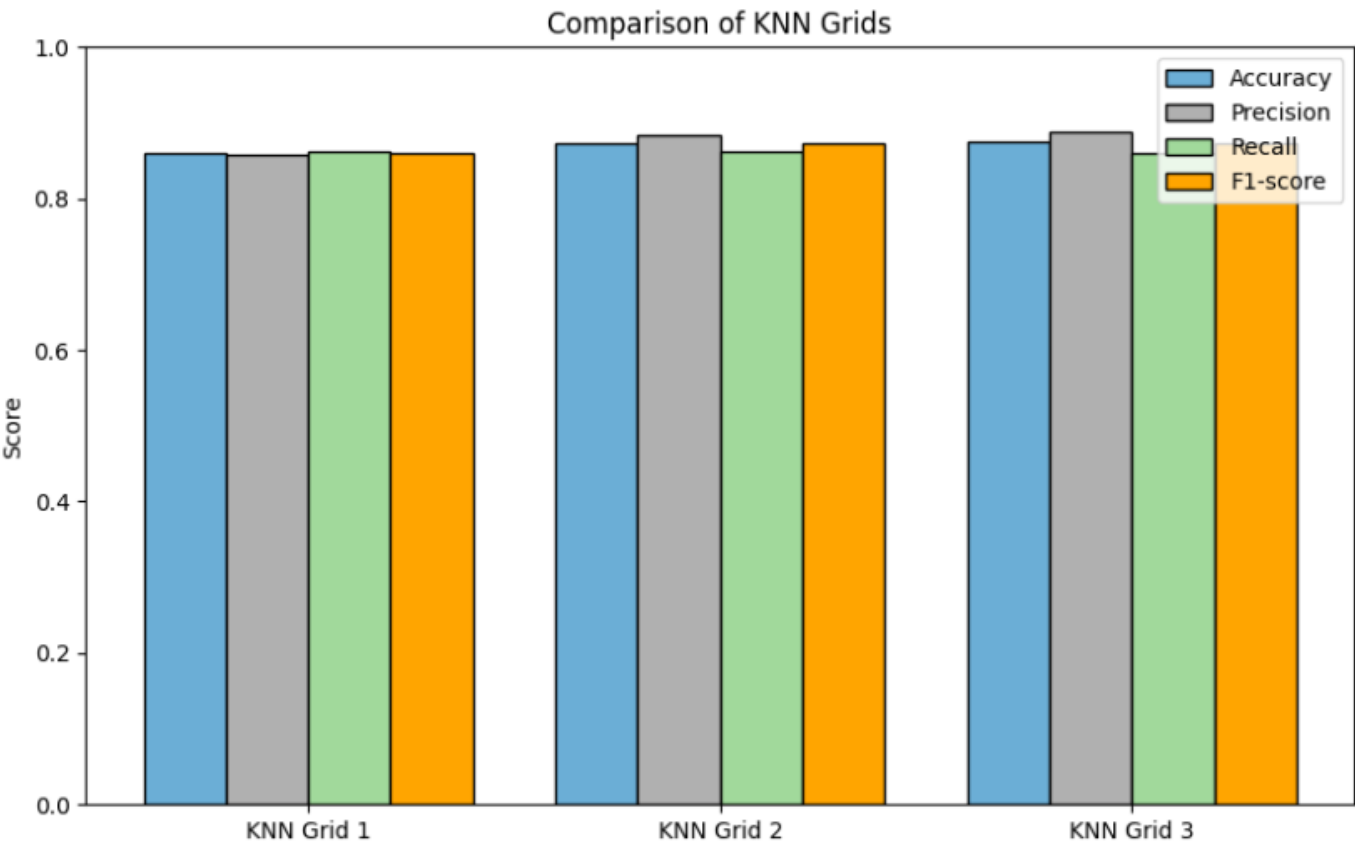
Used Hyperparameters:

```
# Grid 1
param_grid_knn_1 = {
    'n_neighbors': [1, 2, 3],
    'weights': ['uniform'],
    'metric': ['manhattan']
}

# Grid 2
param_grid_knn_2 = {
    'n_neighbors': [5, 7, 9],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

# Grid 3
param_grid_knn_3 = {
    'n_neighbors': [11, 13, 15],
    'weights': ['distance'],
    'metric': ['euclidean']
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	KNN Grid 1	0.859000	0.857305	0.862192	0.859741
1	KNN Grid 2	0.873643	0.882842	0.862334	0.872468
2	KNN Grid 3	0.875286	0.887973	0.859627	0.873570



Step 1: Look at F1-score (most balanced metric for class imbalance)

- KNN Grid 1 → 0.859741
- KNN Grid 2 → 0.872468
- KNN Grid 3 → 0.873570 (**highest F1-score**)

Step 2: Check Accuracy

- KNN Grid 1 → 0.859000
- KNN Grid 2 → 0.873643
- KNN Grid 3 → 0.875286 (**slightly higher than the others**)

➤ **KNN Grid 3** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

5. Random Forest

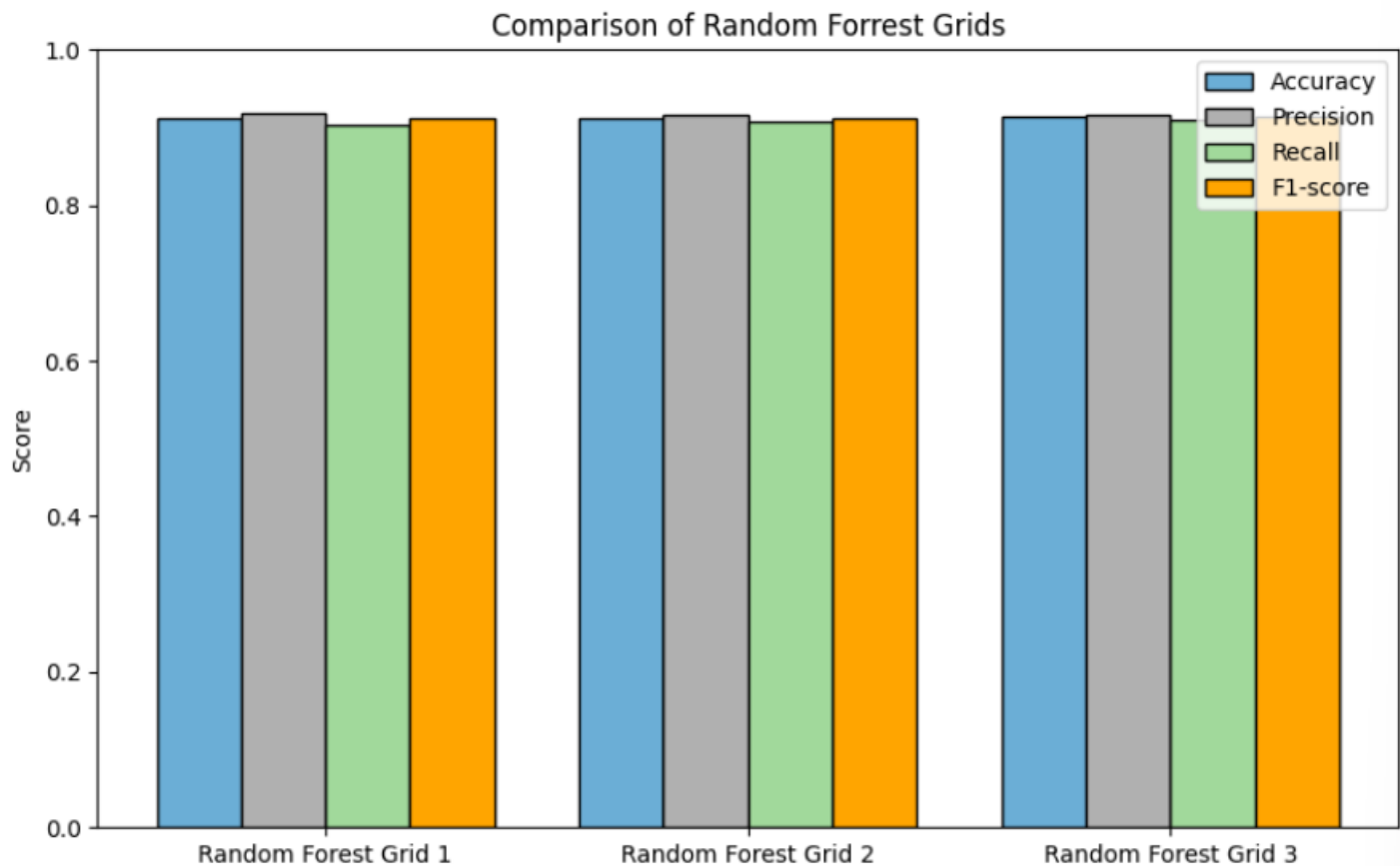
Used Hyperparameters:

```
# Grid 1
param_grid_rf_1 = {
    'n_estimators': [10, 30, 50],
    'max_features': ['log2']
}

# Grid 2
param_grid_rf_2 = {
    'n_estimators': [100, 150],
    'max_features': ['sqrt', 'log2']
}

# Grid 3
param_grid_rf_3 = {
    'n_estimators': [200, 300, 500],
    'max_features': ['sqrt', 'log2']
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	Random Forest Grid 1	0.911214	0.918931	0.903139	0.910966
1	Random Forest Grid 2	0.912571	0.916870	0.908536	0.912684
2	Random Forest Grid 3	0.913000	0.917061	0.909246	0.913136



Step 1: Look at F1-score (most balanced metric for class imbalance)

- Random Forest Grid 1 → 0.910966
- Random Forest Grid 2 → 0.912684
- Random Forest Grid 3 → 0.913136 (**highest F1-score**)

Step 2: Check Accuracy

- Random Forest Grid 1 → 0.911214
- Random Forest Grid 2 → 0.912571
- Random Forest Grid 3 → 0.913000 (**slightly higher than the others**)

➤ **Random Forest Grid 3** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

6. Multi-Layer Perceptron (MLP) Neural Network

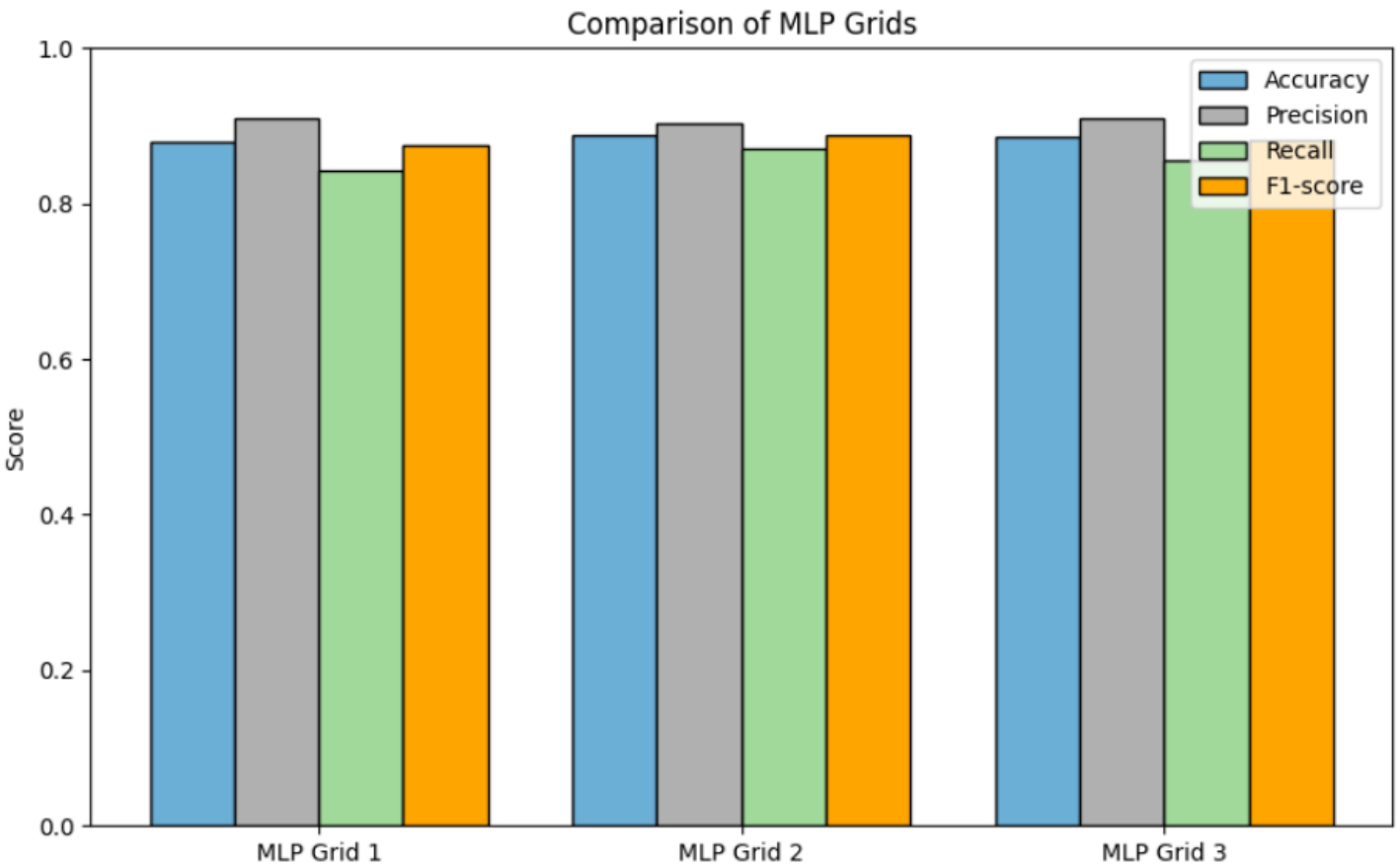
Used Hyperparameters:

```
# Grid 1
param_grid_mlp_1 = {
    'hidden_layer_sizes': [(30,), (50,)],
    'activation': ['tanh'],
    'alpha': [0.01, 0.1],
    'learning_rate_init': [0.0001]
}

# Grid 2
param_grid_mlp_2 = {
    'hidden_layer_sizes': [(50,), (100,), (50,50)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.001, 0.0001],
    'learning_rate_init': [0.001]
}

# Grid 3
param_grid_mlp_3 = {
    'hidden_layer_sizes': [(100,), (100,50), (150,100)],
    'activation': ['relu'],
    'alpha': [0.0001],
    'learning_rate_init': [0.005, 0.01]
}
```

	Model	Accuracy	Precision	Recall	F1-score
0	MLP Grid 1	0.879357	0.909342	0.843380	0.875120
1	MLP Grid 2	0.889071	0.903664	0.871598	0.887341
2	MLP Grid 3	0.885286	0.909366	0.856491	0.882137



Step 1: Look at F1-score (most balanced metric for class imbalance)

- MLP Grid 1 → 0.875120
- MLP Grid 2 → 0.887341 (**highest F1-score**)
- MLP Grid 3 → 0.882137

Step 2: Check Accuracy

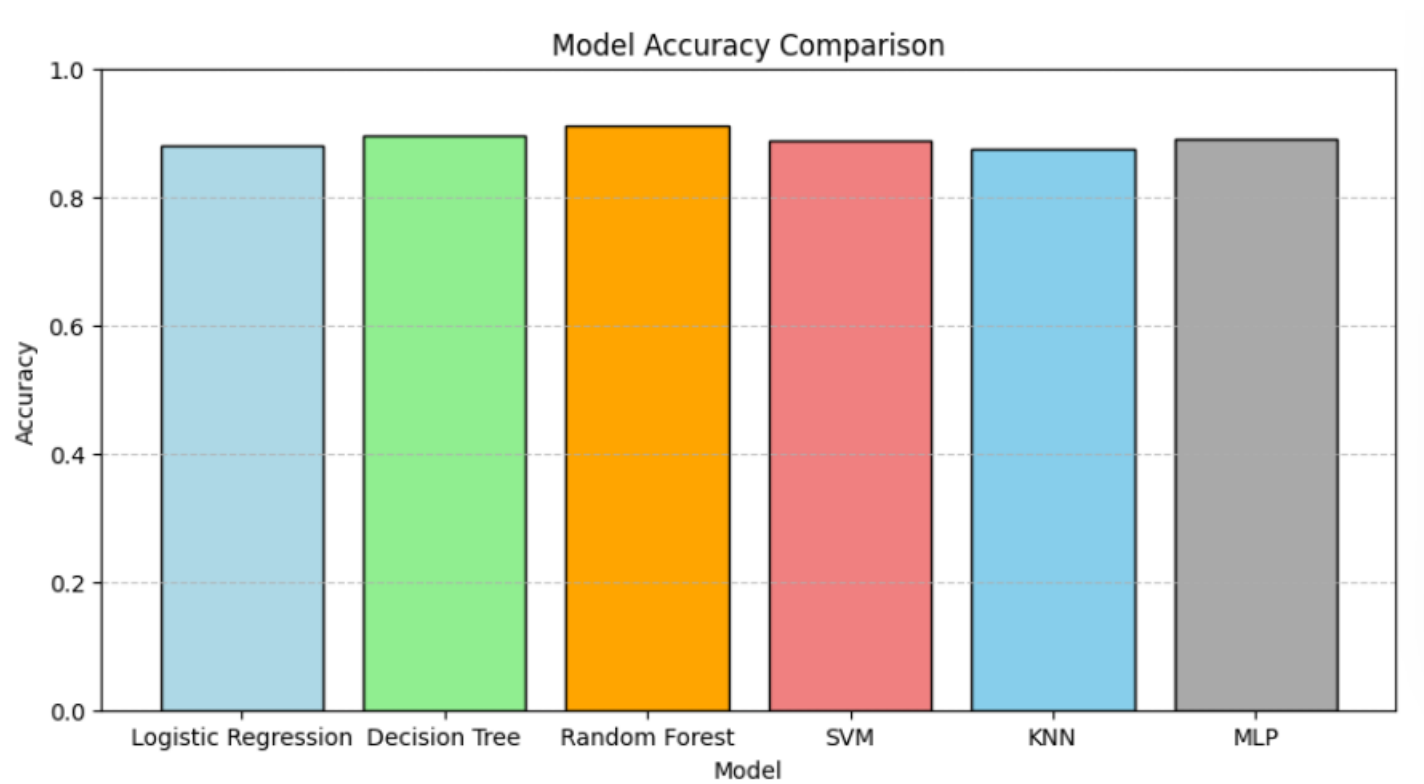
- MLP Grid 1 → 0.879357
- MLP Grid 2 → 0.912571 (**slightly higher than the others**)
- MLP Grid 3 → 0.885286

➤ **MLP Grid 2** is the **best model** because it has the **highest F1-score**, **highest accuracy**, and **balanced precision and recall**.

Model Comparison and Selection

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.879214	0.912996	0.838963	0.874415
1	Decision Tree	0.895071	0.875169	0.922189	0.898064
2	Random Forest	0.910500	0.915633	0.904803	0.910186
3	SVM	0.887929	0.908396	0.863474	0.885366
4	KNN	0.875286	0.887973	0.859627	0.873570
5	MLP	0.889071	0.903664	0.871598	0.887341

Compare Using a bar graph:



- As seen in the **table** and the corresponding **bar graph**, the **Random Forest** model consistently achieves the best performance across key metrics. With the **highest F1-score (0.910186)** and **AUC (0.910500)**, it outperforms the other models in both the numerical comparison and the visual representation. Hence, **Random Forest** is selected as the **best model** for loan approval prediction.